

Личный тест · не бенчмарк · для себя · v4.3 (7 связок, тяжёлые задачи)

Харнесс решает больше, чем модель

7 связок «харнесс × модель» на 4 тяжёлых creation-задачах со строгим скрытым оракулом. В этой выборке все дошли до правильного результата, поэтому дальше сравниваются затраты. Одна и та же GPT-5.6-sol получила разброс оценки от \$3,8 до \$23 в зависимости от обвязки.

ВЕРДИКТ (для личного выбора инструмента)

Оставаться на Codex. В этом срезе он дешевле и быстрее всех; opencode рядом; зафиксированная конфигурация omp кратно дороже.

Главное число. На одной и той же модели gpt-5.6-sol medium восемь прогонов получили оценки **\$3,77 в Codex**, **\$5,28 в opencode** и **\$23,06 в omp**. Датасет даёт **6,11×**; с поправкой на известный недосчёт cache-write консервативное дно равно **5,38×**.

Корректность в этой выборке не разделила участников. 56 из 56 прогонов (7 связок × 4 задачи × 2) прошли видимый и скрытый оракул. Восемь успехов на связку не доказывают равенство качества, но дают право сравнить затраты успешных прогонов.

Что и на чём гоняли

Ранняя версия теста гоняла 6 лёгких задач — там все давали 100%, разделения не было (это и был главный упрёк). Их **выкинул**. Здесь — только 4 **авторские bespoke creation-задачи**, специально тяжёлые и с большим скрытым слоем на краевые случаи:

t7 · TS формулы

движок формул таблицы: токенайзер + парсер приоритетов + функции + строгая таксономия ошибок. 61+45 тестов.

t8 · PY запросы

движок запросов: filter→sort→distinct→paginate→project, строгая семантика типов. 29+22.

t9 · TS стек-VM

интерпретатор ассемблера: стек, переменные, метки, переходы, лимит шагов. 30+33.

t10 · PY cron

планировщик cron: разбор 5 полей, правило OR для dom/dow, следующие N срабатываний. 19+22.

Оракул честный: заранее проверено, что «услужливое» решение (коэрсит типы, читает неинициализированную переменную как 0, ставит AND вместо OR) **падает** на скрытом слое; эталон проходит; для t10 совпадение с независимым brute-force сканером на 24 расписаниях. Грейдинг без установки зависимостей: TS — `node --test + tsc`, PY — `unittest`.

СЕМЬ СВЯЗОК

● **codex56**

Codex · 5.6-sol · med

дешевле всех

● **codex55**

Codex · 5.5 · high

● **oc56**

opencode · 5.6-sol · med

● **oc55**

opencode · 5.5 · high

● **omp55**

omp · 5.5 · high

● **сcorpus**

Claude Code · opus-4.8 · xhigh

● **omp56**

omp · 5.6-sol · med

дороже всех

Все на подписках, поэтому \$ — **нормализованная API-оценка**, а не списание с карты. Для codex/opencode использована формула \$5/\$0,5/\$30 за Mtok input/cache-read/output; cache-write по текущему тарифу стоит \$6,25/M, но отдельные токены записи не сохранились. Поэтому оценки codex/oc — нижние, а разброс 6,11х имеет консервативное дно 5,38х. сcorpus стоит особняком: это другая модель.

1. Корректность: 56 из 56 в этой выборке

Каждая связка прошла оба слоя оракула на всех 4 задачах в обоих прогонах. Ни одного провала — даже на 15-минутных многошаговых задачах со скрытыми краевыми случаями, заточенными ловить «услужливость».

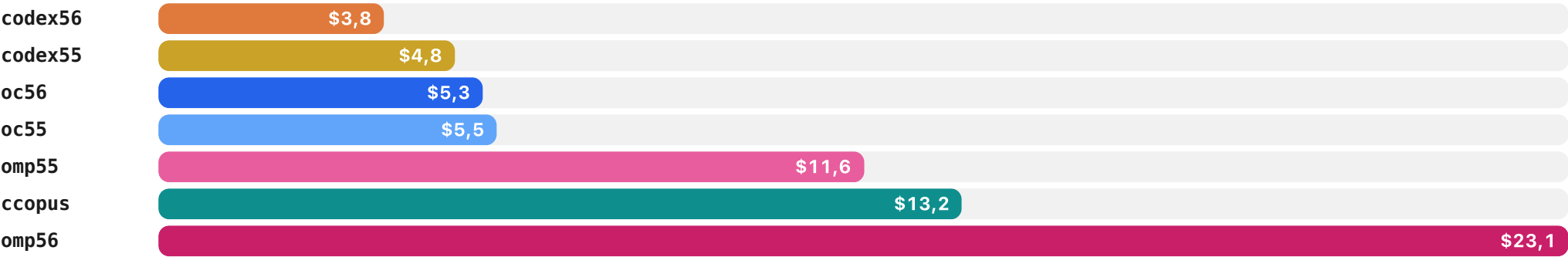
	t7 формулы	t8 запросы	t9 VM	t10 cron
● codex56	✓✓	✓✓	✓✓	✓✓
● codex55	✓✓	✓✓	✓✓	✓✓
● oc56	✓✓	✓✓	✓✓	✓✓
● oc55	✓✓	✓✓	✓✓	✓✓
● omp55	✓✓	✓✓	✓✓	✓✓
● omp56	✓✓	✓✓	✓✓	✓✓
● сcorpus	✓✓	✓✓	✓✓	✓✓

Разделения по правильности в 56 прогонах нет. Это условие сравнения затрат, а не доказательство равного качества harness.

2. Цена: полная таблица (нормализованная оценка, \$ за 2 прогона)

ЗАДАЧА	CODEX56	CODEX55	OC56	OC55	OMP55	CCOPUS	OMP56
t7 формулы	1,36	1,82	1,47	2,04	3,23	5,68	6,72
t8 запросы	0,80	1,09	1,24	1,16	1,72	2,68	5,81
t9 VM	0,75	0,75	1,05	1,31	3,81	2,13	5,02
t10 cron	0,86	1,11	1,51	0,99	2,82	2,75	5,51
Итого · 8 прогонов	3,77	4,77	5,28	5,50	11,58	13,24	23,06

Зелёным — самый дешёвый на задаче (везде Codex). Время — в разделе 3: оно оказалось той же осью, что и цена.



Итого за 8 прогонов. Для codex/oc это нижние оценки из-за недосчитанного cache-write; даже консервативная поправка оставляет разрыв omp56/Codex кратным, не ниже 5,38x.

3. Время: та же ось, что и цена

Сначала я его выкинул — счёл, что при параллельном запуске оно зашумлено конкуренцией. Ошибся: шум есть, но сигнал сильнее шума. **Порядок связей по времени совпал с порядком по цене — все семь позиций** (Pearson $r = 0,986$). С одной оговоркой, которую разбираю ниже: половина времени codex — реконструкция, и одна граница в этом ряду не доказана.

ЗАДАЧА · СЕК (ПРОГОН 1 / ПРОГОН 2)	● CODEX56	● CODEX55	● OC56	● OC55	● OMP55	● CCOPUS	● OMP56
t7 формулы	213 / 244°	254 / 292°	204 / 205	327 / 339	371 / 311	905 / 705	508 / 1153
t8 запросы	75 / 102°	104 / 111°	172 / 248	193 / 172	149 / 163	346 / 390	591 / 810
t9 VM	126 / 95	121 / 114	125 / 166	192 / 170	277 / 395	174 / 212	650 / 173
t10 cron	169 / 124	138 / 225	200 / 290	160 / 138	193 / 469	359 / 343	794 / 394
Итого · 8 прогонов, мин	19,1°	22,6°	26,8	28,2	38,8	57,2	84,5

° — 8 ячеек codex по t7/t8 не замерены, а **восстановлены** по таймстемпам рабочих каталогов (scratchpad с замерами вычистился из /private/tmp). Это **нижняя оценка**: на 8 ячейках, где сохранился и замер, и таймстемпы, реконструкция занижала на **10–65 с (в среднем 40)**. Отсюда неприятное, но честное: **55% итогового времени codex56 и 56% времени codex55 — реконструкция, а не замер**. Именно у лидера таблицы цифра времени проверена хуже всех. Устойчивость — ниже. Прогоны шли параллельно (–P4/–P6), абсолютные секунды несут шум конкуренции.

0,986

корреляция цены и времени по 7 связкам; ранги совпали — шесть позиций из семи устойчивы

\$0,20–0,30

за минуту по зафиксированной формуле; codex/oc немного недосчитаны из-за cache-write

4,4x

разрыв по времени на одной модели: Codex 19 мин · opcode 27 · omp 85

В этом стенде дорогие связки одновременно работали дольше. Причинность не изолирована: время и цена могут быть общими следствиями длинных циклов. Секундомер здесь полезен как ранний сигнал кратного перерасхода, а не как универсальная формула цены.

НАСКОЛЬКО ЭТО ВЫДЕРЖИВАЕТ НЕДОМЕР ВРЕМЕНИ У CODEX

Раз половина времени codex — реконструированная нижняя оценка, проверим совпадение рангов на прочность, накинув поправку на все четыре восстановленные ячейки:

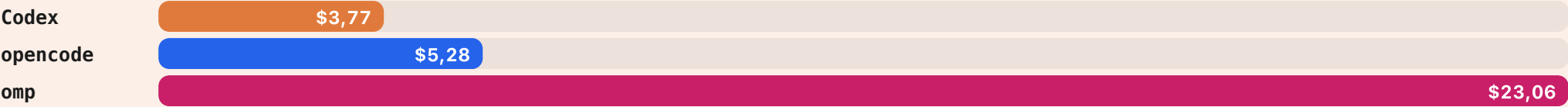
- как замерено — codex56 19,1 · codex55 22,6 · oc56 26,8 · oc55 28,2 · omp55 38,8 · scopus 57,2 · omp56 84,5 → ранги совпадают с ценой полностью;
- +40 с на ячейку (средняя недооценка) — codex56 21,7 · codex55 25,2 → порядок тот же, **совпадение держится**;
- +65 с на ячейку (худшая мыслимая, максимум из замеренных, разом на все четыре) — codex55 становится 27,0 против 26,8 у oc56 и **они меняются местами**. Совпадение рангов ломается — на 9 секунд.

Значит честная формулировка такая: **шесть позиций из семи устойчивы, граница codex55 ↔ oc56 не доказана** — она внутри ошибки реконструкции. Крупные разрывы (omp56 вчетверо дольше Codex, scopus втрое) поправкой в минуту не двигаются вообще. Общий вывод «время идёт той же осью, что цена» на этой паре не держится и не должен — он держится на масштабе разрывов.

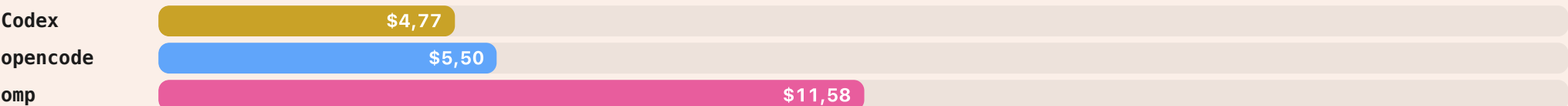
4. Главный инсайт: одна модель, три harness, разброс 5,4–6,1x

Это ядро теста. Берём **одну и ту же** модель и меняем только обвязку. Корректность везде та же — а счёт разный в разы.

GPT-5.6-SOL · MEDIUM — ТА ЖЕ МОДЕЛЬ, 8 ПРОГОНОВ



GPT-5.5 · HIGH — ТА ЖЕ МОДЕЛЬ, 8 ПРОГОНОВ



Обе шкалы в одном масштабе. В этом личном срезе harness оказался сильным рычагом расхода. Для профиля 5.6 medium точный спред лежит между 5,38× и 6,11×: нижняя граница учитывает максимально возможный недосчёт cache-write в Codex.

ЭТО НЕ ШУМ ВЫБОРКИ: РАЗРЫВ ДЕРЖИТСЯ НА КАЖДОЙ ЗАДАЧЕ

ЗАДАЧА	CODEX	OPENCODE	OMP	OMP / CODEX
t7 формулы	\$1,36	\$1,47	\$6,72	4,92×
t8 запросы	\$0,80	\$1,24	\$5,81	7,25×
t9 VM	\$0,75	\$1,05	\$5,02	6,69×
t10 cron	\$0,86	\$1,51	\$5,51	6,44×
итого	\$3,77	\$5,28	\$23,06	6,11×

Все четыре задачи используют одну модель `gpt-5.6-sol medium`. Значения 4,9–7,3× рассчитаны по зафиксированной формуле и служат верхними оценками: cache-write недосчитан в дешёвой колонке Codex. Консервативная поправка по сумме всё равно оставляет минимум 5,38×.

5. Где именно живёт эта цена

Сам собой напрашивается ответ «дорогие harness просто болтливей». **Данные говорят обратное.** Разложим счёт на стоимость output tokens и остаток, который output не объясняет.

СВЯЗКА	\$ ВСЕГО	\$ OUTPUT	NON-OUTPUT RESIDUAL	ДОЛЯ RESIDUAL	OUTPUT TOKENS
codex56	3,77	1,70	2,07	55%	56 741
ос56	5,28	1,32	3,96	75%	43 962
ссорpus	13,24	5,12	8,12	61%	204 874
omp56	23,06	2,97	20,09	87%	99 067

Residual = итоговая оценка минус стоимость output tokens. Датасет не позволяет разложить его между свежим input, cache writes, cache reads, служебным контекстом и числом ходов.

Решающая пара — codex56 и ос56. opencode выдаёт **на 23% меньше** output tokens, чем Codex (43 962 против 56 741), и получает на 40% большую оценку по зафиксированной формуле. Болтливости спред не объясняет. Точный механизм residual установить нельзя без первичных input/cache-счётчиков.

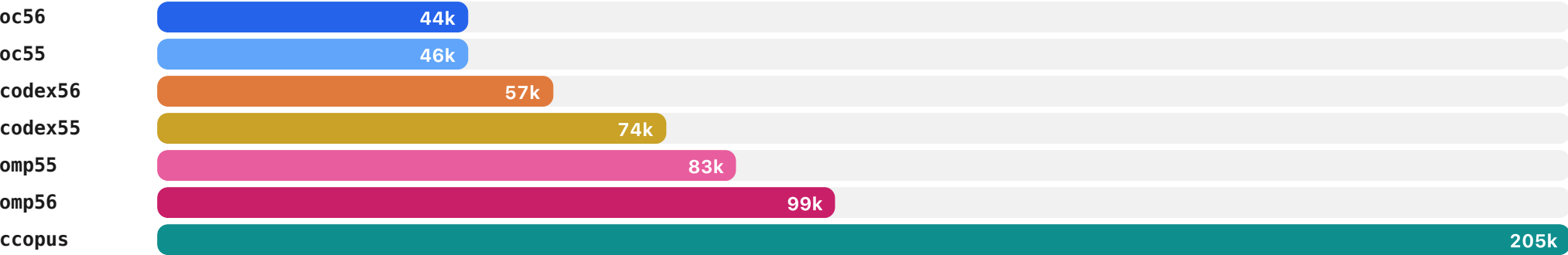
У omp56 output объясняет \$2,97 из \$23,06; residual составляет \$20,09, или 87%. Это показывает границу данных: видимый объём ответа объясняет только малую часть итоговой оценки.

ДВА ПРОФИЛЯ МЕНЯЮТСЯ МЕСТАМИ ПРИ СМЕНЕ HARNESS

omp56 (5.6-sol, medium) получает оценку **\$23,06**, почти вдвое больше omp55 (5.5, high, \$11,58). В Codex порядок обратный: 5.6-medium дешевле 5.5-high (\$3,77 против \$4,77). Модель и effort меняются одновременно, поэтому эксперимент не изолирует причину. Он показывает, что два полных профиля `model+effort` не сохраняют единый порядок цены при смене harness.

6. opencode: почти как Codex, и самый немногословный

opencode находится рядом с Codex по зафиксированной оценке и выдаёт **меньше всех output tokens**. После максимально консервативной поправки cache-write точный порядок Codex/opencode может сбиться, но оба остаются далеко от omp56.



Выходные токены за 8 прогонов. opencode — самый терпеливый; ccorpus (opus) жжёт в ~4,5× больше — премиум за многословность, не за результат.

7. Методология и честные поправки

ПОПРАВКА К ПРОШЛОЙ ВЕРСИИ: «CODEX56 ДИКО ПРЫГАЕТ» — СНЯТО

В прошлом черновике я написал, что codex56 нестабилен по цене (\$0,6–3,1). **Это был артефакт.** Флаг `-c mcp_servers={}` в этой версии Codex **не** отключает MCP из глобального конфига — flaky `exa` MCP периодически висел и раздувал сессии в 2–6×. Настоящий фикс — отдельный `CODEX_HOME` без MCP. На чистых прогонах codex56 — самый ровный и дешёвый. Цифры выше все пересняты начисто.

- **Изоляция обвязок.** Codex — чистый `CODEX_HOME` без MCP. `opencode` — `--pure` (без внешних плагинов/MCP). `omp` — `--no-lsp`, провайдер `openai-codex`. Каждый прогон проверен на web/MCP-вызовы: реальных нет (одно совпадение по подстроке «еха» в схеме оказалось ложным — вызова не было).
- **Цена приведена к общей шкале с известной погрешностью.** Codex/opencode пересчитаны по формуле $\$5/\$0,5/\$30$; cache-write должен стоить $\$6,25/M$, но отдельные токены не сохранились. Поэтому их значения — нижние оценки. `omp` считает цену сам; у `opencode` цена в логах нулевая (OAuth). Fallback `ccusage` $\$0,125$ снят арифметикой: один output той ячейки стоит $\$0,1596$, верное значение $\$0,4479$.
- **Автономность под ключ.** Все 56 прогонов — headless, без единого вопроса к человеку: Codex (sandbox+never), Claude Code (scoped-права), `opencode` (`--auto`), `omp` (`-p`). Все довели до зелёного сами.
- **Время — и поправка к моей же оговорке.** В прошлой версии я время не показал: «зашумлено параллельным запуском». Проверил — **зря выкинул**: ранги по времени совпали с рангами по цене на всех семи связках ($r = 0,986$), шум конкуренции крупные разрывы не съедает. Замер — от старта процесса до выхода, при `-P4/-P6`. 8 ячеек codex по `t7/t8` — восстановленная нижняя оценка (занижает на 10–65 с, в среднем 40): **55% итогового времени codex56 — реконструкция, а не замер**, см. раздел 3 и проверку на устойчивость там же. Переснимать 8 прогонов не стал: они шли бы при другой загрузке и были бы менее сравнимы с остальной таблицей, а не более.
- **Выборка.** 4 задачи × 2 прогона на связку — **индикативно, не доказательно**. Доверять крупным разрывам (цена в разы), а не микро-дельтам. Эфорты не выровнены — это «мои реальные конфиги».

Что делаю по итогу

- **Остаюсь на Codex.** Дешевле всех при той же корректности, самый лаконичный цикл. `codex56` (5.6-sol med) — рабочий дефолт.
- **opencode — годная альтернатива.** Рядом по цене, самый немногословный; если понадобится его UX/фичи — переплата минимальна.
- **omp — не для моей рутины по цене.** Конфигурация 16.5.2 + `gpt-5.6 medium` оказалась самой дорогой и медленной на каждой задаче. Другие версии и ценность дополнительных функций тест не измеряет.
- **Claude Code + Opus** оказался дорогим и долгим в моём наборе. Другая модель и effort не позволяют использовать эту строку как чистое сравнение harness.

Окружение: OpenAI Codex v0.144.4 · `opencode` 1.17.17 · `omp` 16.5.2 · Claude Code v2.1.210 · Node 26.5.0 · Python 3.14.6 · TS 5.9.3. 56 прогонов (7 связок × 4 задачи × 2), метрики восстановлены из логов харнессов (`ccusage-codex` + пересчёт по токенам, `claude-json`, потоки событий `opencode/omp`). Не бенчмарк — личная история «что лучше именно мне на моём стеке». v4.3: публичный репозиторий очищен до `t7-t10` и актуального датасета; удалены эталонные решения и старые артефакты; 6,11× помечен верхней оценкой, 5,38× — консервативным дном с учётом cache-write; `input/output` переименован в проверяемый `output/residual`.